

10 — Schleifen

Wiederholungen automatisieren mit while und for

HTW Berlin – Wirtschaftsingenieurwesen

SoSe 2026

Agenda

1. Wozu Schleifen?
2. `while` – Wiederholen mit Bedingung
3. Endlosschleifen vermeiden
4. `for` – über Sammlungen iterieren
5. `range()` – Zahlenfolgen
6. `break` und `continue`
7. List Comprehensions

Lernziel: Wiederkehrende Aufgaben (Bestand prüfen, Summen bilden, Listen filtern) automatisieren – statt 50 mal copy-paste.

Wie wir heute arbeiten: Nach jedem Konzept zeigt die Folie *Live-Demo* (ich im Terminal) und *Sofort ausprobieren* (Sie im Notebook 10).

Wozu Schleifen?

Ohne Schleife

```
print(warenkorb[0])  
print(warenkorb[1])  
print(warenkorb[2])      # was, wenn 50 Einträge?
```

Mit Schleife

```
for produkt in warenkorb:  
    print(produkt)
```

DRY wieder einmal – gleiche Logik einmal schreiben, beliebig oft anwenden.

while - Wiederholen mit Bedingung

```
versuche = 3
while versuche > 0:
    print(f"Restversuche: {versuche}")
    versuche = versuche - 1      # auch: versuche -= 1
print("Keine Versuche mehr.")
```

- Solange die Bedingung **wahr** ist, läuft der Block
- Bedingung wird **vor** jedem Durchlauf geprüft
- **Wichtig:** irgendetwas im Block muss die Bedingung am Ende **falsch** machen

► **Live-Demo** - 01_while_countdown.py

📎 **Sofort ausprobieren** - Notebook 10, Kap. While: Countdown 10 $\rightarrow$ 0, Hochzählen 0 $\rightarrow$ 5, gerade Zahlen

Endlosschleifen vermeiden

```
versuche = 3
while versuche > 0:
    print("...")
    # versuche -= 1    # vergessen -> Endlosschleife
```

- Bei laufender Endlosschleife: **STRG+C** im Terminal
- In Jupyter: “Interrupt Kernel”

while True + break - bewusst eingesetzt

while True:

```
    eingabe = input("Modell? ")
```

```
    if eingabe in ["Eco-Sneaker", "Hemp-High", "Bambus-Boot"]:  
        break
```

```
    print("nicht im Sortiment, nochmal.")
```

Faustregel: jede Schleife muss eine **Abbruchbedingung** haben – sei es im Header oder über break.

for - über Sammlungen iterieren

```
warenkorb = ["Eco-Sneaker", "Hemp-High", "Bambus-Boot"]
```

```
for produkt in warenkorb:  
    print(produkt)
```

- for VAR in SAMMLUNG: – in jedem Durchlauf bekommt VAR den nächsten Wert
- Funktioniert mit **Listen, Tupeln, Sets, Strings, Dicts**
- Beendet sich automatisch, wenn alle Elemente abgearbeitet sind

► **Live-Demo** - 02_for_produkte.py

📎 **Sofort ausprobieren** - Notebook 10, Kap. For: Liste ausgeben, gegen Schwellenwert prüfen, Strings durchgehen

range() - Zahlenfolgen erzeugen

```
for i in range(5):           # 0, 1, 2, 3, 4  
    print(i)
```

```
for i in range(1, 6):       # 1, 2, 3, 4, 5  
    print(i)
```

```
for i in range(0, 21, 5):    # 0, 5, 10, 15, 20  
    print(i)
```

Form	Bedeutung
range(stop)	0, 1, ..., stop-1
range(start, stop)	start, ..., stop-1
range(start, stop, step)	mit Schrittweite

Summen, Mittelwerte - klassische for-Muster

```
preise = [89.95, 109.00, 135.50]
```

```
# Summe
```

```
gesamt = 0
```

```
for p in preise:
```

```
    gesamt = gesamt + p
```

```
# Mittelwert
```

```
mittel = gesamt / len(preise)
```

```
print(f"Gesamt: {gesamt:.2f} EUR, Mittel: {mittel:.2f} EUR")
```

- “Sammelvariable” (gesamt) **vor** der Schleife auf den Startwert
- In jedem Durchlauf erweitern
- Nach der Schleife auswerten

► **Live-Demo** - 03_for_range_summe.py

break und continue

```
for produkt in warenkorb:  
    if produkt == "Bambus-Boot":  
        break                # Schleife komplett verlassen  
    print(produkt)
```

```
for preis in preise:  
    if preis > 100:  
        continue            # diesen Durchlauf überspringen  
    print(f" güns t i g: {preis}")
```

Anweisung	Wirkung
break	Schleife sofort beenden
continue	Rest des Durchlaufs überspringen, nächster Wert

break / continue - Demo & Übung

► **Live-Demo** - 04_break_continue.py

✎ **Sofort ausprobieren** - Notebook 10, Kap. break/continue: Suche abbrechen, ungerade Zahlen überspringen

Schleife über ein Dictionary

```
preise = {"Eco-Sneaker": 89.95, "Hemp-High": 109.00, "Bambus-Boot
```

```
for name, preis in preise.items():  
    print(f"{name}: {preis} EUR")
```

Methode	Was kommt im Durchlauf
for k in d:	nur Schlüssel
for v in d.values():	nur Werte
for k, v in d.items():	Paare

► **Live-Demo** - 05_for_dict_warenkorb.py

List Comprehensions - kompakter for

Klassisch

```
brutto = []  
for p in [89.95, 109.00, 135.50]:  
    brutto.append(p * 1.19)
```

Comprehension

```
brutto = [p * 1.19 for p in [89.95, 109.00, 135.50]]
```

Mit Filter

```
günstig = [p for p in preise if p < 100]
```

Comprehensions sind **kürzer**, aber nur sinnvoll, wenn der Ausdruck **eine Zeile** bleibt. Im Zweifel: klassische for-Schleife.

Cheat Card

Aufgabe	Syntax
while-Schleife	<code>while cond: ...</code>
for über Liste	<code>for x in liste: ...</code>
Zahlenfolge	<code>for i in range(start, stop, step):</code>
Dict iterieren	<code>for k, v in d.items():</code>
Schleife verlassen	<code>break</code>
Durchlauf überspringen	<code>continue</code>
Liste bauen	<code>[ausdruck for x in liste if cond]</code>

Faustregel: feste Anzahl Wiederholungen → `for`. Unbekannte Anzahl bis Ereignis → `while`.

Ausblick: Notebook 11 - Fehlerbehandlung

Schleifen + Eingaben + Daten = **Dinge gehen schief:**

```
for eintrag in eingaben:
    zahl = int(eintrag)           # ValueError, wenn keine Zahl
    print(1 / zahl)             # ZeroDivisionError, wenn 0
```

Im nächsten Notebook: **try/except** – Programme robust gegen Fehler machen, statt sie crashen zu lassen.

Heute geübt

- ✓ while – bedingte Wiederholung
- ✓ Endlosschleifen erkannt und vermieden
- ✓ for über Listen, Strings, Dicts
- ✓ range() für Zahlenfolgen
- ✓ break / continue – Schleifensteuerung
- ✓ List Comprehensions als Kurzform

Zur Vertiefung im Notebook 10:

- “Sofort ausprobieren”-Aufgaben in jedem Kapitel
- Praktische Übung: Bestandsaufnahme mit `while True + break`
- Trainingsmaterial: gestufte Aufgaben (Quadratzahlen, FizzBuzz, Filterung)